

15.3.5 Sinks

How do we perform output using our `Process` type? We'll often want to send the output of a `Process[IO,O]` to some *sink* (perhaps sending a `Process[IO,String]` to an output file). Somewhat surprisingly, we can represent a sink as a process that *emits functions*:

```
type Sink[F[_],O] = Process[F[_], O => Process[F,Unit]]
```

This makes a certain kind of sense. A `Sink[F[_], O]` provides a sequence of functions to call with the input type `O`. The function returns `Process[F,Unit]`. Let's look at a `Sink` that writes strings to a file:

```
def fileW(file: String, append: Boolean = false): Sink[IO,String] =
  resource[FileWriter, String => Process[IO,Unit]]
    { IO { new FileWriter(file, append) }}
    { w => constant { (s: String) => eval[IO,Unit](IO(w.write(s))) }}
    { w => eval_(IO(w.close)) }
```

```
def constant[A](a: A): Process[IO,A] = ← The infinite, constant stream
  eval[IO,A](IO(a)).repeat
```

That was easy. And notice what *isn't* included—there's no exception handling code here—the combinators we're using guarantee that the `FileWriter` will be closed if exceptions occur or when whatever is feeding the `Sink` signals it's done.

We can use `tee` to implement a combinator `to`, a method on `Process` which pipes its output to a `Sink`:

```
def to[O2](sink: Sink[F,O]): Process[F,Unit] =
  join { (this zipWith sink)((o,f) => f(o)) }
```



EXERCISE 15.12

The definition of `to` uses a new combinator, `join`, defined for any `Process`, which concatenates a nested `Process`. Implement `join` using existing primitives. This combinator should be quite familiar to you from previous chapters.

```
def join[F[_],O](p: Process[F, Process[F,O]]): Process[F,O]
```

Using `to`, we can now write programs like the following:

```
val converter: Process[IO,Unit] =
  lines("fahrenheit.txt").
  filter(!_ startsWith("#")).
  map(line => fahrenheitToCelsius(line.toDouble).toString).
  pipe(intersperse("\n")).
  to(fileW("celsius.txt")).
  drain
```